

Output-Procedure Call Diagnostic test

Name (caps) _____ Student ID (9 digits)

Lab Time _____ Desk Number: Desk- _____

How to work out what a code fragment does: draw a table

The simplest and most reliable way to work out what a code fragment does (and to understand it) is to draw a table. This is called tracing or dry running code. Let's work out what this fragment does:

```
System.out.print("Holding out for");           // L1
System.out.println("a hero");                  // L2
System.out.println("is by Bonnie Tyler.");      // L3
```

Executing Line L1

This prints out the given string. Important to notice is it uses `System.out.print` not `System.out.println`. This means anything that follows goes on the same line with no spaces unless there is one in the String (there isn't here)

LINE	printed
L1	Holding out for

Executing Line L2

This prints on the same line as the previous text printed but at the end goes onto a new line so that anything else printed in later lines will go to a new line. No spaces are printed after "for".

LINE	printed
L1	Holding out for
L2	a hero

Executing Line L3

This now prints the new string on the new line. As it uses `println` the cursor finishes at the start of the next line.

LINE	printed
L1	Holding out for
L2	a hero
L3	is by Bonnie Tyler.

Overall what is printed is:

```
Holding out for
a hero
is by Bonnie Tyler
```

Notice how the title of the song has been printed on one line and the artist on the next, but that a space is missing - we have found a bug! We need to add an explicit space in the Line L1 String.

Output-Procedure Call Diagnostic test

You must show all your working in the following. No correct table (exactly the same as the solution)... no marks. If you do not get them all right and do not understand why, ask for help.

Complete the dry run table for each fragment of code given in the questions below. **Give** the value(s) printed to the screen when each is executed.

Follow the module dry run style guide in your answers.

1.

```
System.out.print("I am ");           // L1
System.out.println("the one and only"); // L2
```

Complete the dry run table (you may not need all rows):

LINE	printed

Write EXACTLY what is printed here

2.

```
System.out.println("Sisters are doin' it"); // L1
System.out.println("for themselves");       // L2
```

Complete the dry run table (you may not need all rows):

LINE	printed

Write EXACTLY what is printed here

Output-Procedure Call Diagnostic test

3.

```
System.out.print("I will survive ");           // L1
System.out.println("by");                       // L2
System.out.println("Gloria Gaynor");            // L3
```

Complete the dry run table (you may not need all rows):

LINE	printed

Write EXACTLY what is printed here

4.

```
System.out.println("I will survive");           // L1
System.out.print("by");                         // L2
System.out.println("Gloria Gaynor");            // L3
```

Complete the dry run table (you may not need all rows):

LINE	printed

Write EXACTLY what is printed here

Output-Procedure Call Diagnostic test

How to work out what a procedure call does: draw more tables

With procedures you draw a new table for each procedure call. Consider:

```
public static void printBornToRun ()
{
    System.out.print("Born to Run");           // L1
    return;
}
public static void printSong ()
{
    System.out.print("Springsteen wrote ");    // L2
    printBornToRun();                          // L3
    return;
}
```

with a test call `printSong ();`

Executing Line L2

Notice here we execute line L2 first as we call the method `printSong` which just prints a string.

printSong()	
LINE	printed
L2	Springsteen wrote

Executing Line L3

We have to pause filling out the table and start a new table for `printBornToRun`. Once it has finished executing we will be able to return to complete the original table.

printSong()	
LINE	printed
L2	Springsteen wrote
L3	

printBornToRun()	
LINE	printed

Executing Line L1

As we are now executing the procedure we execute Line L1. which prints Born to Run then returns back to where we came from so Line L2.

printBornToRun()	
LINE	printed
L1	Born to run

Output-Procedure Call Diagnostic test

Continuing Executing Line L3

As we are now executing the original procedure again so return to the printSong table. The previous table has shown us that the call to printBornToRun printed out Born to Run so we note that that is what Line L3 prints

printSong()	
LINE	printed
L2	Springsteen wrote
L3	Born to run

We therefore end up with the final two tables:

printSong()	
LINE	printed
L2	Springsteen wrote
L3	Born to run

printBornToRun()	
LINE	printed
L1	Born to run

Overall what is printed is:

Springsteen wrote Born to Run

Now you try...

PTO

Output-Procedure Call Diagnostic test

5.

```
public static void songName1 ()
{
    System.out.println("All you need is love");    // L1
    return;
}
public static void printFamousSong1 ()
{
    songName1();    // L2
    System.out.println("was written by");    // L3
    System.out.println("Lennon and McCartney");    // L4
    return;
}
```

with a test call `printFamousSong1 ();`

Complete the dry run tables (you may not need all rows):

printfamousSong1()	
LINE	printed

songName1()	
LINE	printed

Write EXACTLY what is printed here

Output-Procedure Call Diagnostic test

6.

```
public static void songName2 ()
{
    System.out.println("All you need is love");    // L1
    return;
}
public static void printFamousSong2 ()
{
    System.out.println("was written by");          // L2
    songName2();                                   // L3
    System.out.println("Lennon and McCartney");    // L4
    return;
}
```

with a test call printFamousSong2 ();

Complete the dry run tables (you may not need all rows):

printfamousSong2()	
LINE	printed

songName2()	
LINE	printed

Write EXACTLY what is printed here